

Guía de Ayuda y Pistas para el Examen

PARTE 1: Procesamiento Distribuido

Pista 1: Cómo simular dificultad en tareas

```
def process_task(self, task_id, difficulty, is_process=False):
    """
    Simula procesamiento con diferente dificultad
    difficulty: 1 (fácil) a 5 (difícil)
    """

    print(f"Iniciando tarea {task_id} (dificultad: {difficulty})")

    # La dificultad determina el tiempo de procesamiento #
    # Fácil (1): 0.3 segundos, Difícil (5): 1.5 segundos
    processing_time = difficulty * 0.3
    time.sleep(processing_time) # Simula trabajo

    # Resultado basado en dificultad
    result = task_id * difficulty

    print(f"Tarea {task_id} completada - Resultado: {result}")
    return result
```

Pista 2: Locks para hilos vs procesos

```
# Para HILOS (comparten memoria)
self.thread_lock = threading.Lock()

with self.thread_lock:
    self.tasks_completed_threads += 1

# Para PROCESOS (memoria separada)
self.tasks_completed_processes = multiprocessing.Value('i', 0)

with self.tasks_completed_processes.get_lock():
    self.tasks_completed_processes.value += 1
```

Pista 3: Crear y manejar hilos

```
def run_with_threads(self, tasks):
```

```
threads = []

for task_id, difficulty in tasks:
    # Crear hilo para cada tarea
    thread = threading.Thread(
        target=self._worker_thread, # Función que ejecutará el hilo
        args=(task_id, difficulty)  # Argumentos para la función
    )
    threads.append(thread)
    thread.start() # Iniciar hilo

# Esperar a que todos terminen
for thread in threads:
    thread.join()
```

PARTE 2: Almacenamiento Distribuido

CONFIGURACIÓN DOCKER-COMPOSE.YML

Archivo básico YAML que necesitas:

```
version: '3.8'

services:
  mongo1:
    image: mongo:latest
    container_name: mongo_node1
    ports:
      - "27017:27017" # 📌 Puerto para conectar desde Python
    networks:
      - mongo-network

  mongo2:
    image: mongo:latest
    container_name: mongo_node2
    ports:
      - "27018:27017" # 📌 Puerto diferente para el segundo nodo
    networks:
      - mongo-network

networks:
  mongo-network:
    driver: bridge
```

Pasos para ejecutar:

```
# 1. Navegar a la carpeta donde está docker-compose.yml
cd part2-distributed-storage

# 2. Levantar los contenedores (funcionarán en segundo plano)
docker-compose up -d

# 3. Verificar que estén ejecutándose
docker ps

# 4. Para detenerlos cuando termines
docker-compose down
```

DISTRIBUCIÓN AUTOMÁTICA DE DOCUMENTOS

Pista: Cómo decidir en qué nodo guardar cada documento

```
def _select_node_for_document(self, document_data):  
    """  
    Decide en qué nodo MongoDB guardar el documento  
    Usa 'sharding' por hash para distribución uniforme  
    """  
    # Obtener ID del documento (o generarlo si no existe)  
    document_id = str(document_data.get('id', len(document_data)))  
  
    # Crear hash del ID para distribución consistente  
    hash_value = hashlib.md5(document_id.encode()).hexdigest()  
  
    # Convertir hash a número y usar módulo para elegir nodo  
    node_index = int(hash_value, 16) % 2 # %2 porque tenemos 2 nodos  
  
    if node_index == 0:  
        return self.db1, "node1"  
    else:  
        return self.db2, "node2"
```

Ejemplo de uso:

```
def insert_document(self, data):  
    # 1. Seleccionar nodo automáticamente  
    target_db, node_name = self._select_node_for_document(data)  
  
    # 2. Preparar documento con metadatos  
    document = {  
        '_id': data.get('id'),  
        'data': data,  
        'node': node_name, # Guardamos en qué nodo quedó  
        'created_at': datetime.now()  
    }  
  
    # 3. Insertar en el nodo seleccionado  
    result = target_db.documents.insert_one(document)  
    print(f"Guardado en {node_name}")
```

BÚSQUEDA DISTRIBUIDA

Pista: Buscar en TODOS los nodos

```
def find_document(self, document_id):
    """
    Busca un documento en todos los nodos MongoDB
    """
    results = []

    # Buscar en el PRIMER nodo
    if self.db1:
        doc1 = self.db1.documents.find_one({'_id': document_id})
        if doc1:
            doc1['source_node'] = 'node1' # Marcar de dónde vino
            results.append(doc1)

    # Buscar en el SEGUNDO nodo
    if self.db2:
        doc2 = self.db2.documents.find_one({'_id': document_id})
        if doc2:
            doc2['source_node'] = 'node2' # Marcar de dónde vino
            results.append(doc2)

    return results
```

Pista: Conectar a múltiples MongoDB

```
def __init__(self):
    # Conexión al PRIMER MongoDB (puerto 27017) self.client1
    = MongoClient('mongodb://localhost:27017') self.db1 =
    self.client1['distributed_db']

    # Conexión al SEGUNDO MongoDB (puerto 27018) self.client2
    = MongoClient('mongodb://localhost:27018') self.db2 =
    self.client2['distributed_db']
```

GENERAR DATOS DE PRUEBA

Pista: Crear documentos de ejemplo

```
def generate_sample_data(num_documents=100):  
    sample_data = []  
    for i in range(num_documents):  
        sample_data.append({  
            'id': i, # ID único para cada documento  
            'name': f'Documento_{i}',  
            'value': i * 10,  
            'category': f'categoria_{i % 5}', # 5 categorías diferentes  
            'timestamp': datetime.now()  
        })  
    return sample_data
```

FLUJO RECOMENDADO PARA RESOLVER EL EXAMEN

Gestión del Tiempo:

1. **Primera hora:** Parte 1 (Procesos vs Hilos)
2. **Segunda hora:** Parte 2 (MongoDB Distribuido)

Checklist Parte 1:

- Implementar `process_task()` con `time.sleep()`
- Usar `threading.Lock()` para hilos
- Usar `multiprocessing.Value()` para procesos
- Crear y ejecutar hilos correctamente
- Crear y ejecutar procesos correctamente
- Medir y comparar tiempos

Checklist Parte 2:

- Crear `docker-compose.yml`
- Conectar a ambos MongoDB desde Python
- Implementar distribución por hash
- Hacer búsquedas en ambos nodos
- Generar 100 documentos de prueba
- Mostrar estadísticas de distribución

SOLUCIÓN DE PROBLEMAS COMUNES

Error: No se puede conectar a MongoDB

Verificar que Docker esté ejecutándose

```
docker ps
```

Si no hay contenedores, ejecutar:

```
docker-compose up -d
```

Verificar en Python con:

```
try:
```

```
    client = MongoClient('mongodb://localhost:27017', serverSelectionTimeoutMS=5000)
```

```
    client.admin.command('ping') # Test de conexión
```

```
    print("Conectado a MongoDB")
```

```
except Exception as e:
```

```
    print(f"Error: {e}")
```

Error: Puerto ya en uso

bash

Buscar qué está usando el puerto

```
netstat -ano | findstr :27017
```

O detener todos Los contenedores MongoDB

```
docker stop $(docker ps -q)
```

```
docker-compose down
```

Error: Documentos duplicados

Usar _id único para cada documento

```
document = {
```

```
    '_id': data.get('id'), # ID Debe ser único
```

```
    'data': data,
```

```
    # ... otros campos
```

```
}
```

RECUERDA:

1. **¡No te bloques!** Si una parte no funciona, pasa a la siguiente
2. **Prueba incrementalmente** - verifica cada función pequeña antes de integrar
3. **Los errores son oportunidades** de aprendizaje
4. **Usa esta guía** como referencia, no como solución completa